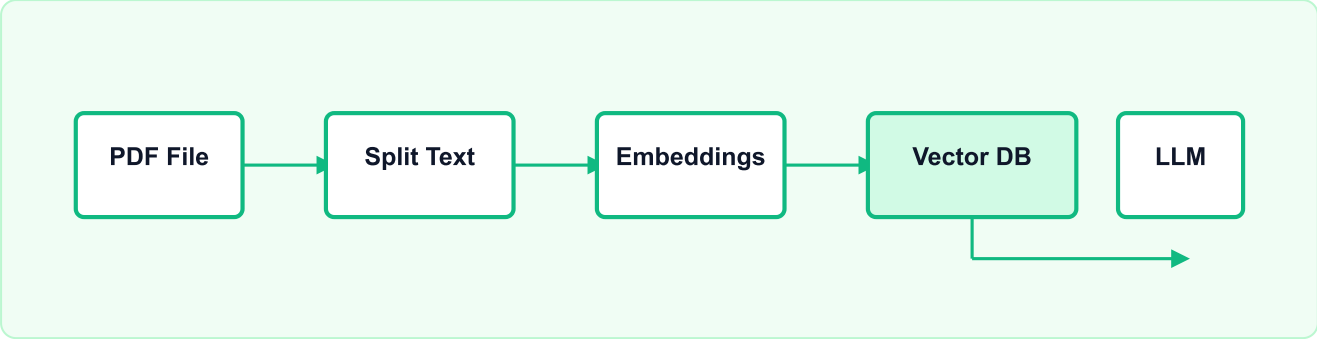
XÂY DỰNG CHATBOT RAG ĐỌC HIỂU PDF VỚI PYTHON

Retrieval-Augmented Generation (RAG) là kiến trúc giúp Chatbot trí tuệ nhân tạo trả lời câu hỏi dựa trên tài liệu cụ thể của bạn, thay vì dựa vào kiến thức chung dễ dẫn đến tình trạng sinh thông tin ảo (hallucination). Tài liệu này hướng dẫn chi tiết cách dùng LangChain, OpenAI và ChromaDB để xây dựng hệ thống.



Bước 1: Cài đặt thư viện

Mở terminal và cài đặt hệ sinh thái LangChain cùng các package phụ trợ để xử lý PDF và cơ sở dữ liệu Vector. Đảm bảo bạn đã cài đặt Python 3.9+.

```
      bash pip install langchain langchain-openai langchain-chroma langchain-community
pypdf
```

Bước 2: Load và Split dữ liệu

Vì AI có giới hạn bộ nhớ ngữ cảnh (context window), chúng ta phải cắt file PDF thành các đoạn nhỏ (chunk) khoảng 1000 ký tự. Có sự chồng lấp (overlap) 200 ký tự để không làm đứt đoạn ý nghĩa.

```
import os from langchain_community.document_loaders import PyPDFLoader from
langchain_text_splitters import RecursiveCharacterTextSplitter # Khai báo API Key
os.environ["OPENAI_API_KEY"] = "sk-api-key-cua-ban" # 1. Đọc file PDF loader =
PyPDFLoader("tai_lieu.pdf") docs = loader.load() # 2. Cắt nhỏ văn bản text_splitter =
RecursiveCharacterTextSplitter( chunk_size=1000, chunk_overlap=200 ) chunks =
text_splitter.split_documents(docs)
```

Bước 3: Tạo Vector Embeddings

Biến các đoạn text thành các ma trận số thực (vector) để máy tính có thể so sánh khoảng cách ngữ nghĩa. Chúng ta sẽ lưu các vector này vào **ChromaDB**, một cơ sở dữ liệu vector chạy local (không cần cài đặt server).

```
from langchain_openai import OpenAIEmbeddings from langchain_chroma import Chroma # Khởi tạo mô hình Embedding embeddings = OpenAIEmbeddings() # Đưa dữ liệu vào Database vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings) # Tạo bộ truy xuất dữ liệu (k=3 nghĩa là lấy 3 đoạn text liên quan nhất) retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

Bước 4: Thiết lập Prompt và Kết nối LLM

Khởi tạo mô hình sinh ngôn ngữ (GPT-4o-mini) và viết một "System Prompt" (Chỉ thị hệ thống) chặt chẽ để ép AI chỉ trả lời dựa trên những gì tìm được, không tự suy diễn.

```
from langchain_openai import ChatOpenAI from langchain.chains import create_retrieval_chain from langchain.chains.combine_documents import create_stuff_documents_chain from langchain_core.prompts import ChatPromptTemplate # Khởi tạo LLM với temperature=0 (Câu trả lời ổn định, không bay bổng) llm = ChatOpenAI(model="gpt-4o-mini", temperature=0) # Viết Prompt system_prompt = ( "Bạn là một trợ lý AI hữu ích. " "Sử dụng ngữ cảnh dưới đây để trả lời. " "Nếu không tìm thấy đáp án, hãy nói 'Tôi không biết', không bịa thông tin.\n\n" "Ngữ cảnh:\n{context}" ) prompt = ChatPromptTemplate.from_messages([ ("system", system_prompt), ("human", "{input}"), ]) # Kết nối (Chain) Retriever và LLM qa_chain = create_stuff_documents_chain(llm, prompt) rag_chain = create_retrieval_chain(retriever, qa_chain)
```

Bước 5: Đặt câu hỏi và thực thi

Khi gọi hàm `invoke()`, hệ thống ngầm thực hiện 3 bước: Nhúng câu hỏi thành vector -> Tìm các chunk gần giống nhất trong ChromaDB -> Nhét câu hỏi & các chunk đó vào Prompt -> Đưa cho OpenAI xử lý và trả về text.

```
# Đặt câu hỏi thực tế cho tài liệu question = "Những điều khoản bảo hành chính được nhắc đến là gì?" response = rag_chain.invoke({"input": question}) print("🤖 Câu trả lời của AI:") print(response["answer"])
```

💡 **Kinh nghiệm triển khai thực tế (Production):**

1. **Xử lý File phức tạp:** `PyPDFLoader` chỉ phù hợp với PDF thuần text. Nếu file của bạn có bảng biểu (Table), biểu đồ, hay chia 2 cột, hãy tìm hiểu công cụ nâng cao như **Unstructured.io** hoặc **LlamaParse**.

2. **Vector Storage:** Đối với dự án nhỏ, ChromaDB local là đủ. Nhưng nếu scale lên cho hàng ngàn user hoặc hàng GB tài liệu, hãy sử dụng các VectorDB chuyên dụng trên Cloud như **Pinecone, Qdrant**, hoặc **pgvector** (PostgreSQL).

3. **Bảo mật:** Nếu tài liệu cực kỳ bảo mật (tài chính, y tế), hãy thay thế OpenAI bằng các LLM chạy local (như Llama 3, Qwen) thông qua **Ollama** để đảm bảo dữ liệu không bao giờ rời khỏi máy chủ của công ty.